

Project Proposal: Mini Autonomous Vehicle Using Computer Vision

Real-World Problem

Autonomous vehicles must continuously interpret visual information from their surroundings and make decisions based on that information. This project addresses a simplified version of the autonomous-driving perception problem: enabling a low-cost robotic vehicle to navigate a controlled road-like environment while responding appropriately to lanes, obstacles, and traffic signs. Correct recognition alone is not sufficient; the vehicle must also determine where objects are located relative to its path. For example, detecting a pedestrian is useful only if the system can accurately localize the pedestrian and determine whether the person is in the vehicle's path. A reliable vision system is therefore important for making safe navigation decisions.

Proposed Solution

I will build a small autonomous vehicle using a Raspberry Pi, camera, motors, motor driver, and ultrasonic distance sensor. The camera will provide the primary visual input. However, if after sufficient efforts, managing hardware becomes too much of a hassle, I will switch to a simulation environment. Classical computer vision methods such as image preprocessing and lane detection will be combined with a lightweight YOLO object detection model to identify and localize objects such as vehicles, pedestrians, and stop signs. The object detector will output both a class label and a bounding box for each detected object. A control system will combine lane information, object locations, and distance-sensor measurements to determine whether the vehicle should move forward, slow down, stop, or turn. The final system will be tested on a controlled track containing lanes, obstacles, and traffic signs.

Dataset

The project will use the publicly available BDD100K dataset, a large-scale driving video dataset containing real-world road images and annotations for tasks including object detection and road-scene understanding. It includes annotated road objects such as cars, pedestrians, traffic signs, and other relevant categories. The dataset will provide training and/or evaluation images for the object-detection component.

Dataset URL: <https://www.vis.xyz/bdd100k/>

Evaluation Metrics

The object-detection model will be evaluated using metrics that measure both recognition and localization. The primary metric will be mean Average Precision (mAP), including mAP at an Intersection over Union threshold of 0.50 (mAP@0.50). IoU measures the overlap between a predicted bounding box and the ground-truth bounding box, calculated as the intersection area divided by the union area. A prediction will be considered a correct detection only when the predicted class is correct and the IoU is at least 0.50. This prevents a model from receiving full credit for identifying an object while placing its bounding box in the wrong location. Precision and recall will also be reported using the same IoU criterion.

In addition, real-time deployment will be evaluated using inference latency, measured in milliseconds per frame, and frames per second (FPS) on the Raspberry Pi / Simulation environment. The complete vehicle will be assessed using navigation success rate, defined as the percentage of test runs that complete a predefined course without leaving the lane or failing to respond correctly to required obstacles or stop signs.

Technical Approach and Rigor

The project will be developed incrementally. Initial work will establish camera capture, motor control, and the vehicle's sensor pipeline/ simulation environment setup. Next, lane-detection and image-processing methods will be implemented. The BDD100K images and annotations will then be prepared for object detection, and a lightweight YOLO model will be trained or fine-tuned and evaluated on held-out data. Model performance will be analyzed using IoU-based detection metrics rather than classification accuracy alone. Different model sizes or configurations may be compared to investigate the trade-off between localization accuracy and computational efficiency on edge hardware. Finally, the perception system will be integrated with vehicle control and tested repeatedly on a controlled track. If hardware development becomes impractical, the same perception pipeline can be evaluated in a simulated driving environment while preserving the computer-vision focus.

Timeline and Deliverables

Weeks 1–3: Hardware/simulation setup, camera input, and motor or vehicle-control pipeline

Weeks 4–6: Lane detection and image-processing pipeline

Weeks 7–9: BDD100K preparation and YOLO object detection

Weeks 10–12: IoU-based evaluation, model comparison, and autonomous-control integration

Weeks 13–14: Testing, optimization, and collection of final metrics

Week 15: Final demonstration, GitHub documentation, and report

GitHub

Source code, preprocessing scripts, model configuration, experiments, evaluation results, and final documentation will be maintained in a GitHub repository.